

When Do Simple Graph Features Fail on Structured Link Anomalies?

COMP5331 project type: Research

Group number: [Group number]

Member 1: Student ID []; name []; research/FYP supervisor [name or none]; own research/FYP topic [topic or none]; difference from this project [explanation or not applicable].

Member 2: Student ID []; name []; research/FYP supervisor [name or none]; own research/FYP topic [topic or none]; difference from this project [explanation or not applicable].

Member 3: Student ID []; name []; research/FYP supervisor [name or none]; own research/FYP topic [topic or none]; difference from this project [explanation or not applicable].

Member 4: Student ID []; name []; research/FYP supervisor [name or none]; own research/FYP topic [topic or none]; difference from this project [explanation or not applicable].

Member 5: Student ID []; name []; research/FYP supervisor [name or none]; own research/FYP topic [topic or none]; difference from this project [explanation or not applicable].

Member 6 (if applicable): Student ID []; name []; research/FYP supervisor [name or none]; own research/FYP topic [topic or none]; difference from this project [explanation or not applicable].

Declaration: This project is conducted solely within COMP5331. It is not work for another course, a research project, or an FYP.

Project description

Background and research question

Many graph anomaly detectors use complex representations, yet the strength of a model depends on what the benchmark calls anomalous. Latapy and Rajeh show that simple features of link streams combined with classical learning can perform well when anomalous interactions are introduced randomly. Randomly chosen endpoints may be easy to identify because they violate ordinary patterns of endpoint activity, temporal recurrence, or local graph structure. An adversarial or naturally arising suspicious interaction need not be so conspicuous. A link can occur between locally plausible nodes, at a typical time, and with the normal frequency of interaction while still being unusual in a more subtle sense. The related work by Poštuvan and colleagues on learning-based link anomaly detection in continuous-time dynamic graphs motivates more careful task and evaluation definitions.

Our research question is whether the conclusion about simple features remains valid as injected anomalies become structurally and temporally plausible. We will test a sequence of controlled anomaly mechanisms and examine the performance of the same feature families under each. The primary hypothesis is that random-link anomaly detection overstates performance on structured anomalies, particularly when injected interactions preserve node activity and community membership. A secondary hypothesis is that selecting a small group of complementary temporal and structural features using only training data can recover some performance at lower extraction cost, even if it cannot fully remove the gap. Both hypotheses are falsifiable: if simple features remain equally effective across mechanisms, or feature selection offers no benefit over a fixed small set, we will report that outcome.

Prior work and proposed contribution

The main reference is Matthieu Latapy and Stephany Rajeh, “Trivial Graph Features and Classical Learning are Enough to Detect Random Anomalies,” ICDM 2025. We will use its author implementation where possible for event preprocessing, anomaly injection, feature construction, and classical prediction. Poštuvan et al., “Learning-Based Link Anomaly Detection in Continuous-Time Dynamic Graphs,” TMLR 2024, will guide our definition of dynamic links, baseline evaluation, and threats to validity. The proposed contribution is a mechanism-controlled stress test of the random-anomaly conclusion: a graded collection of anomaly generators, a mechanism-stratified evaluation, and a cost-aware feature-budget analysis under a common experimental protocol.

The setting is a time-ordered link stream of events (`source`, `destination`, `timestamp`). We will first generate a synthetic stream with communities, heterogeneous node activity, and recurring interactions. The generator will expose ground-truth structural parameters, so we can change only one difficulty factor at a time. We will create at least four anomaly conditions: random endpoint replacement; replacement within a broad community; replacement that also matches endpoint activity bins; and temporal-burst anomalies placed within a plausible local activity period. The latter conditions should be harder because they deliberately preserve features that trivial random replacements disturb. We will inspect each generator's distributions before using it as evidence. In particular, event timestamps, degrees, and train/test positions must not reveal the injected label through an unintended artifact.

Position among dynamic-graph detectors

The main paper's strong result is specifically about detecting randomly injected links with inexpensive features. Poštuvan and colleagues formulate link anomaly detection in a continuous-time dynamic graph, providing an alternative view of temporal context. MIDAS focuses on microclusters and bursts in edge streams and therefore gives a useful point of comparison for our temporal-burst condition, though its detection target differs from isolated link replacement. These distinctions will shape the hypotheses: a mechanism that preserves node activity and community membership may weaken static graph cues, while a burst detector may retain an advantage on temporally concentrated events. We will keep task definitions visible in every comparison and will not pool incompatible anomaly types into a single performance claim.

Method and evaluation

We will construct a compact set of graph and temporal features available at the moment an event arrives: historical endpoint activity, pair recurrence, elapsed time since recent interactions, local neighborhood overlap, and a small number of neighborhood statistics. Every feature will use only past events in the stream. We will compare logistic regression and random forest or gradient-boosted trees with the same train/validation/test time split across conditions. The full-feature model follows the main paper's spirit. For the feature-budget experiment, we will compare a predefined small baseline with a training-only ranking or forward-selection procedure that incorporates measured extraction cost. We will set a fixed budget in number of features or milliseconds per event; selecting features after seeing test outcomes would invalidate the comparison.

The main metric will be area under the precision-recall curve, because anomalies may be rare. We will also report ROC-AUC, recall at a specified false-positive rate, calibration or score distributions where informative, and feature extraction time. Results will be stratified by anomaly mechanism and injection rate, not averaged into a single figure that hides difficulty. Each synthetic configuration will run across several random seeds; means and uncertainty intervals will be shown. A smaller real-data experiment will use the UCI Messages link stream demonstrated in the author's repository, with the source and subset rule recorded. Its role is to test whether patterns seen in synthetic streams persist under a real event distribution, not to claim ground truth for naturally occurring anomalies that have not been verified.

Several controls will distinguish genuine model performance from benchmark shortcuts. We will keep the train/test event timeline fixed while changing only the anomaly mechanism, use matched anomaly prevalence for

primary comparisons, and ensure that feature normalization is fit on training events. We will compare event-time and node-activity distributions between normal and injected examples; if the injected examples are separable by an artifact, the generator will be revised. We will also inspect whether a detected decline results from a narrower positive definition or from changed negative examples. These controls are necessary for interpreting any claim that one anomaly condition is harder than another.

Experimental controls

The released TGF repository supplies preprocessing, injection, feature extraction, learning stages, and example link-stream data. We will first use those examples to establish the paper's random-injection reference result, then add new anomaly generators at the injection stage while keeping the subsequent feature and classifier code fixed. The structured mechanisms will be calibrated to match the random condition in anomaly prevalence, node activity distribution, and event-time distribution. We will compare each positive set with normal events on these observable properties before training a detector. Feature selection will be fit within each training split, with the selected feature names and extraction costs published for every mechanism. This design distinguishes a gain from true complementarity among graph and temporal cues from a gain caused by inadvertently encoding the anomaly generator. On real link streams, the injected labels will be described as synthetic labels applied to real background activity.

Expected outcomes and deliverables

The project will produce code for controlled stream generation, anomaly injection, history-only feature extraction, classical baselines, feature selection, and plots by anomaly mechanism. The report will include the exact event schema, leakage checks, parameter ranges, and reproducible seeds. We will publish the anomaly-generator parameters alongside each result so that difficulty levels are defined by reproducible mechanisms rather than descriptive names alone. We expect a measurable gap between random and structure-preserving anomalies, but its magnitude is an empirical question. If the gap is absent, that would strengthen the simple-feature conclusion within the tested range. If a selected compact feature set performs as well as the full set, we will quantify the resulting compute saving; if not, we will show which feature interactions are lost. Either result answers a useful question about the limits of inexpensive anomaly detection in dynamic graphs.

Papers to read

1. **Main paper:** Matthieu Latapy and Stephany Rajeh. “Trivial Graph Features and Classical Learning are Enough to Detect Random Anomalies.” *IEEE ICDM*, 2025. [arXiv paper](#).
2. **Dynamic-link setting:** Tim Poštuvan, Claas Grohnfeldt, Michele Russo, and Giulio Lovisotto. “Learning-Based Link Anomaly Detection in Continuous-Time Dynamic Graphs.” *Transactions on Machine Learning Research*, 2024. [arXiv paper](#).
3. **Burst-oriented comparator:** Siddharth Bhatia, Bryan Hooi, Minji Yoon, Kijung Shin, and Christos Faloutsos. “MIDAS: Microcluster-Based Detector of Anomalies in Edge Streams.” *AAAI* 34(04):3242–3249, 2020. [Proceedings paper](#).